ARCHITECTURE & INTERVIEW GUIDE

ParaBank UI + API Test Automation Framework

Playwright · TypeScript · Docker · GitHub Actions · Allure. How the framework is designed, which TypeScript, OOP and design-pattern concepts it uses, and how to present it end to end at lead and architect level.

19 + 1	24 → 1	60s → 20s	4
tests + login setup	logins per 3-browser run	Docker suite run time	environments (local, qa, staging, prod)

Repository: github.com/harimandadi/parabank-ui-api-automation
Application under test: ParaBank (parabank.parasoft.com) by Parasoft
Every code excerpt in this guide is taken from the repository; every number was measured on this project.

Contents

1 Project at a glance
2 Architecture
3 TypeScript features used
4 OOP concepts
5 SOLID principles
6 Design patterns
7 Feature-by-feature walkthrough
8 Problems solved (STAR stories)
9 Lead-level presentation
10 Architect-level discussion
11 Interview questions & answers
12 Command cheat sheet & file map

How to use this guide

- **Before the interview:** read sections 1–2 for the big picture, then 3–6 for concepts, using the real code examples as proof.
- **For the pitch:** memorise the 2-minute script in section 9 and practise the 10-minute walkthrough with the repository open.
- **For senior/architect rounds:** section 10 covers trade-offs, scaling and governance; section 8 gives you concrete problem-solving stories.
- **Boxes:** SAY boxes are ready-to-use interview wording. TIP boxes are extra context.

> **Be ready to explain every line.** Interviewers probe specifics: "*why storageState?*", "*what happens on a 429?*", "*how does the fixture fall back?*". If you used AI tools (Claude Code, the Playwright agents) to build parts of it, say so confidently and explain the decisions. Directing AI tools well is a skill teams now look for. Being caught out on a detail you claimed to have written is what hurts.

> **Opening line:** open with the problem and the result ("reliable UI + API coverage against a rate-limited shared server; 20/20 green in CI"), then let the interviewer pull you into the details. Every section of this guide is a place you can go deeper.

1. Project at a glance

The problem

ParaBank is a realistic online-banking demo app (login, registration, accounts, transfers, transaction search, REST API). The goal was a framework that tests **both the UI and the API**, runs **reliably against a shared, rate-limited public server**, can be pointed at **different environments** safely, and gives **reports that managers and developers can both use**.

Scope delivered

Area	What exists
UI tests	Login (4), Register (5), Account Services menu (2), Transfer Funds (3), Find Transactions (3)
API tests	Login endpoint: valid credentials return the customer; invalid credentials return HTTP 400
Framework code	6 page objects, 1 shared component, 1 API client, typed config, custom fixtures, utilities, JSON test data
Environments	`local` (private ParaBank in Docker), `qa`, `staging`, `prod` (data-changing tests skipped)
Execution	Local machine, Docker Compose, GitHub Actions (inside the Playwright container)
Reporting	Playwright HTML report, Allure (steps, Epic/Feature/Story, failure categories, trend history, single-file version)
AI tooling	Playwright test agents: planner, generator, healer, configured for this project

Tech stack

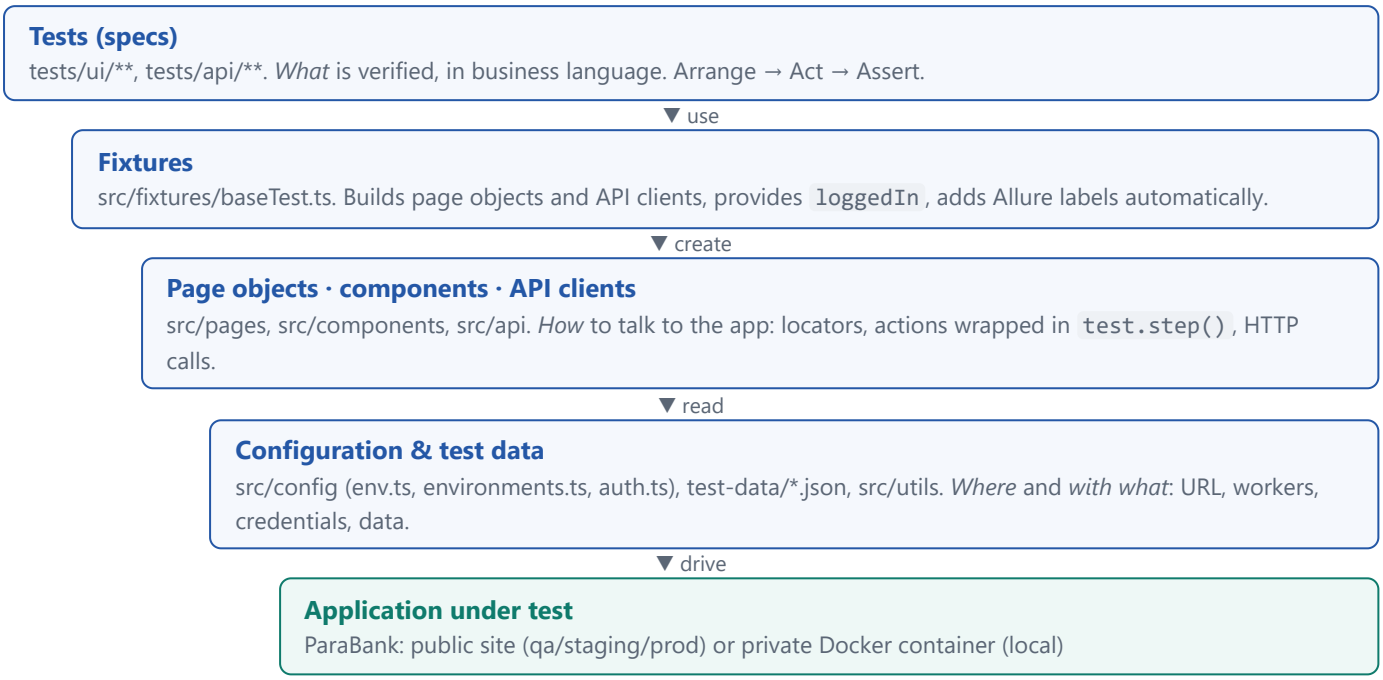
Tool	Why it was chosen
Playwright Test 1.65	One runner for UI and API, auto-waiting locators, built-in parallelism, fixtures, traces, multi-browser
TypeScript (strict)	Catches mistakes before running; typed page objects and config make the framework self-documenting
dotenv + cross-env	Secrets out of code; environment selection that works on Windows, macOS and Linux
Allure	Business-readable reporting with trends and failure categories
Docker / Compose	Reproducible runner, plus a private ParaBank with no rate limit
GitHub Actions	CI on every push and PR, manual runs with an environment picker, report artifacts

Headline results (measured)

Improvement	Before	After
Logins per 3-browser run (Account Services)	24	**1**
Full suite in Docker	~60 s	**~20 s**
Login/Register subset on Windows, 8 runs incl. setup (video on retry only + 127.0.0.1)	32.7 s	**16.3 s**
Full suite runs on the public site	41 failures (rate-limited)	**20/20 green in CI**

2. Architecture

The framework is layered so that each layer has one job and only talks to the layer below it. Tests never contain locators or URLs; page objects never contain assertions about business outcomes; configuration is the only place that knows about environments and secrets.



Cross-cutting pieces

playwright.config.ts

Projects (setup → browsers), per-environment workers, prod filter (`grepInvert`), reporters, failure evidence.

tests/auth.setup.ts

Logs in once, saves cookies to `playwright/.auth/user.json`; browser projects depend on it.

Dockerfile + docker-compose.yml

Runner image from the official Playwright image; private ParaBank service; reports copied out at the end.

.github/workflows/playwright.yml

CI job inside the Playwright container; secrets; environment picker; Allure + HTML artifacts.

Execution flow of one run

1. `playwright.config.ts` imports `ENV`, which loads `.env` / `.env.<env>` and validates `TEST_ENV`.
2. Old Allure results are cleared (main process only).
3. The **setup** project logs in once and saves the session.
4. Browser projects start (Chromium by default; Firefox/WebKit with `ALL_BROWSERS=1`), with workers set per environment.
5. Each test receives its fixtures; Account Services tests start already logged in via the saved session.
6. On failure: screenshot + trace saved (video on retry). Reporters write the HTML report and Allure results.
7. `npm run allure:generate` copies history and builds the full and single-file Allure reports.

> **SAY** *"I separated* **what** *we test from* **how** *we interact with the app and* **where** *we run it. Specs read like test cases, page objects own the locators, and configuration owns environments and secrets. A UI change touches one page object; a new environment is one line in a config map."*

3. TypeScript features used

The project compiles in **strict mode** (`"strict": true` in tsconfig.json), so `null` / `undefined` must be handled explicitly and implicit `any` is not allowed. Below, each feature has a real excerpt and the reason it matters.

3.1 Interfaces and type aliases: contracts for data

src/config/environments.ts

```ts
export type EnvironmentName = 'local' | 'qa' | 'staging' | 'prod';

export interface EnvironmentConfig {
  baseUrl: string;
  allowDataChanges: boolean;
  workers: number;
}
```

Why: every environment must provide the same three fields, and the compiler rejects a typo like `'stagging'`. `CustomerDetails` in RegisterPage.ts does the same for the 11 registration fields.

3.2 String literal union types

`EnvironmentName` above is a union of literals. Only those four strings are valid, so invalid environments are caught at compile time inside the code, and at run time by the type guard (3.6) for values coming from the outside.

3.3 Generics and utility types: `Record`, `Partial`

src/config/environments.ts · tests/ui/register/register.spec.ts

```ts
export const ENVIRONMENTS: Record<EnvironmentName, EnvironmentConfig> = { local: {...}, qa: {...}, staging:
{...}, prod: {...} };

const newCustomer = (overrides: Partial<CustomerDetails> = {}): CustomerDetails => ({
  ...data.customer, username: ENV.registerUsername, password: ENV.registerPassword,
  confirmPassword: ENV.registerPassword, ...overrides,
});
```

Why: `Record<EnvironmentName, …>` forces an entry for *every* environment; forgetting one is a compile error. `Partial<T>` lets a test override only the fields it cares about (e.g. a wrong `confirmPassword`).

3.4 Generic fixtures: `base.extend<Fixtures>`

src/fixtures/baseTest.ts

```ts
type Fixtures = { loginPage: LoginPage; transferFundsPage: TransferFundsPage; loginApi: LoginApi; loggedIn:
void; ... };
export const test = base.extend<Fixtures>({ loginPage: async ({ page }, use) => use(new LoginPage(page)),
... });
```

Why: the generic parameter makes every fixture typed in every test. `async ({ transferFundsPage }) =>` autocompletes, and a misspelled fixture name fails to compile.

3.5 `as const` and indexed access types

src/components/AccountServicesMenu.ts

```ts
export const ACCOUNT_SERVICES_LINKS = ['Open New Account', 'Accounts Overview', 'Transfer Funds', 'Bill Pay', ...] as const;
export type AccountServicesLink = (typeof ACCOUNT_SERVICES_LINKS)[number];

async open(name: AccountServicesLink): Promise<void> { ... }
```

Why: one list is both the **runtime data** (the test checks the menu shows exactly these links, in order) and the **type** (`open('Bil Pay')` is a compile error). A single source of truth, with no duplication.

3.6 User-defined type guards

src/config/environments.ts · src/config/env.ts

```ts
export const isEnvironmentName = (value: string): value is EnvironmentName => value in ENVIRONMENTS;

if (!isEnvironmentName(requestedEnv)) {
  throw new Error(`Unknown TEST_ENV "${requestedEnv}". Use one of: ${Object.keys(ENVIRONMENTS).join(', ')}`);
}
const envName: EnvironmentName = requestedEnv; // narrowed from string, no cast needed
```

Why: environment variables arrive as plain strings. The guard validates at run time *and* narrows the type for the compiler, without an unsafe `as` cast.

3.7 Access modifiers and parameter properties

src/pages/BasePage.ts · src/api/LoginApi.ts

```ts
constructor(protected readonly page: Page) {}            // BasePage: visible to subclasses only
constructor(private readonly request: APIRequestContext) {} // LoginApi: hidden from callers
```

Why: one line declares, assigns and protects the field. `protected` lets every page use `this.page`; `private` means tests can only call `loginApi.login()`, not the raw request object. `readonly` prevents reassigning it by accident.

3.8 Abstract classes

`export abstract class BasePage` cannot be instantiated; it exists only to be extended (see OOP, section 4).

3.9 Getters (accessor properties)

src/config/env.ts

```ts
export const ENV = {
  name: envName,
  baseUrl: process.env.BASE_URL ?? envConfig.baseUrl,
  // Secrets are read lazily, so listing tests or loading the config works without them.
  get username(): string { return required('PARABANK_USERNAME'); },
  get password(): string { return required('PARABANK_PASSWORD'); },
};
```

Why: callers write `ENV.username` like a normal property, but the value is validated when it's first used. `npx playwright test --list` works without secrets, yet a real run fails fast with a clear message.

3.10 Nullish coalescing `??`, template literals, spread and destructuring

- `process.env.TEST_ENV ?? fromRootFile.TEST_ENV ?? 'qa'`: falls back only on `null` / `undefined`, unlike `||`, which would also skip empty strings.
- Template literals build readable step names: `` `Transfer $${amount} from account ${from} to ${to}` ``.
- Spread composes objects: `{ ...data.customer, ...overrides }`.
- Destructuring returns several values cleanly: `const { from, to } = await transferFundsPage.transfer(amount);` and `const [layer, module, file] = path.relative(...).split(path.sep);`

3.11 async/await with typed Promises

Every page action is `async` and returns `Promise<void>` or a typed result, for example `transfer(amount): Promise<{ from: string; to: string }>`. The return type documents what the method gives back, and the compiler rejects using a `Promise` as if it were the resolved value (for example reading `.from` without `await`). Catching every forgotten `await` needs the ESLint rule `no-floating-promises`, which is on the roadmap.

3.12 Modules, JSON imports and strict compiler options

- ES module `import` / `export` keep each file's public surface explicit.
- `"resolveJsonModule": true` lets tests `import data from '../../../test-data/registerData.json'` with types inferred from the JSON.
- `"strict": true` + `"noEmit": true`: the compiler is used purely as a type checker (Playwright transpiles at run time).

3.13 Arrow functions and higher-order functions

Fixtures, helpers (`titleCase`, `uniqueAmount`, `uniqueUsername`) and `test.step(name, async () => {...})` all pass functions as values. `test.step` is a higher-order function that wraps an action with reporting.

> **SAY** *"I lean on the type system to prevent whole classes of bugs: a `Record` forces every environment to be configured, an `as const` list is both test data and a type for menu names, and a type guard turns an untrusted environment variable into a safe, narrowed type."*

4. OOP concepts

4.1 Encapsulation

Each page object bundles its **locators** and the **actions** that use them, and exposes only meaningful operations.

src/pages/TransferFundsPage.ts (simplified)

```
export class TransferFundsPage extends BasePage {
  readonly amountInput: Locator;  readonly fromAccountSelect: Locator;  readonly transferButton: Locator;

  async transfer(amount: string): Promise<{ from: string; to: string }> {
    const ids = await this.accountIds();              // waits for AJAX-loaded dropdowns
    const from = ids[0]; const to = ids[ids.length - 1];
    await test.step(`Transfer $${amount} from account ${from} to ${to}`, async () => { ... });
    return { from, to };
  }
}
```

The test calls `transfer('47.83')` and never knows the dropdowns load asynchronously or which ids the fields have. Secrets are encapsulated too: `ENV.password` hides *where* the value comes from and validates it on access.

4.2 Abstraction

Two levels: `BasePage` is an **abstract class** that captures what every page has in common, and page methods present **business-level operations** (`login()`, `register()`, `findByDateRange()`) instead of clicks and fills.

src/pages/BasePage.ts

```
export abstract class BasePage {
  constructor(protected readonly page: Page) {}
  async open(path: string): Promise<void> { await this.page.goto(`${ENV.appPath}/${path}`); }
}
```

4.3 Inheritance

`LoginPage`, `RegisterPage`, `AccountsOverviewPage`, `TransferFundsPage` and `FindTransactionsPage` all `extends BasePage`. They inherit the protected `page` and the `open()` helper, so `goto()` in each page is one line: `await this.open('transfer.htm')`. The application path (`/parabank`) lives in one place.

> **TIP** The hierarchy is kept deliberately **shallow (one level)**. Deep page hierarchies become fragile; shared behaviour beyond navigation is added through **composition** instead (4.5).

4.4 Polymorphism

Be precise here, because interviewers can tell when it's overstated. In this project polymorphism appears as:

- **Subtype polymorphism:** any page can be used wherever a `BasePage` is expected, and `open()` behaves correctly for each page's own path.
- **Interchangeable implementations behind one contract:** each environment is a different `EnvironmentConfig` object. The rest of the framework treats them identically and behaviour changes through data (URL, workers, safety rule).
- **Fixture overriding:** `test.use({ storageState: AUTH_FILE })` swaps the browser context's behaviour (starting logged in) for a whole spec without changing the tests.

There is no method overriding across pages today because pages have no shared behaviour to specialise. If a natural case appears (for example an `isLoaded()` each page implements differently), that would be the place to add an abstract method.

4.5 Composition over inheritance

The Account Services menu appears on every logged-in page, but it's not inherited. It's a separate **component** (`AccountServicesMenu`) that tests receive as its own fixture and combine with any page. Fixtures themselves compose: `loggedIn` is built from `page`, `loginPage` and `accountsOverviewPage`.

> ***SAY*** *"Pages inherit only what every page truly shares, navigation through an abstract `BasePage`. Everything else is composed: the menu is a component, and fixtures assemble pages, components and API clients per test. That keeps the hierarchy flat and avoids the 'god base page' problem."*

5. SOLID principles

Principle	Where it shows up
Single Responsibility	Specs assert; page objects interact; `env.ts` loads config; `environments.ts` defines environments; `auth.setup.ts` only authenticates; `allure-report.mjs` only builds reports. Each file changes for one reason.
Open/Closed	New environment = new entry in `ENVIRONMENTS`, with no code edits elsewhere. New module = new page object + fixture line. New spec folders get Allure labels automatically, with no changes to the labelling fixture.
Liskov Substitution	Every page subclass can stand in for `BasePage`; none weakens `open()`'s behaviour.
Interface Segregation	Tests request only the fixtures they use (`async ({ transferFundsPage }) =>`); unused objects are never built. Small focused classes instead of one large "App" object.
Dependency Inversion	Page objects receive `Page` and API clients receive `APIRequestContext` through constructors; they never create browsers or HTTP clients themselves. The fixture layer injects them, so the same class works in any context (normal run, saved session, agent seed).

Other principles worth naming

- **DRY:** one menu-link list for data and types; one login fixture; one URL builder in `BasePage`.
- **Fail fast:** unknown `TEST_ENV` and missing secrets stop the run immediately with an actionable message.
- **Test independence:** every data test creates what it needs (unique usernames, unique amounts), so tests run in any order and in parallel.
- **Secure by default:** no secrets in code or in workflow inputs; `prod` blocks data-changing tests automatically.

6. Design patterns

Pattern	Where	Why / what it gives
Page Object Model	`src/pages/*`	One class per page owns locators and actions. A UI change is fixed in one place, and specs read as business steps.
Component Object	`src/components/AccountServicesMenu.ts`	UI that repeats across pages is modelled once and composed, not copied into every page.
Dependency Injection (Playwright fixtures)	`src/fixtures/baseTest.ts`	Tests declare what they need; the framework builds, injects and tears it down. Lazy: only requested fixtures are created.
Factory	Fixture definitions; `newCustomer()`, `uniqueUsername()`, `uniqueAmount()`	Centralised creation of objects and test data. Callers don't know construction details.
Test Data Builder / Object Mother	`newCustomer(overrides)` + `registerData.json`	Valid default customer plus targeted overrides (`{ confirmPassword: 'different_pass' }`) for negative tests.
Facade	`LoginApi`; page methods like `transfer()`	A simple interface over a complex sequence: HTTP details, headers and encoding, or four UI interactions plus waiting.
Singleton (module)	`ENV` in `src/config/env.ts`	Config is loaded and validated once per process and shared by everything that imports it.
Strategy (configuration-driven)	`ENVIRONMENTS` map	Each environment is a strategy object (URL, workers, data rule) chosen at run time by `TEST_ENV`.
Decorator-style wrapper	`test.step()` around page actions	Adds reporting (named, timed steps in HTML and Allure) around behaviour without changing it.
Guard / Fail-fast	`required()`, `isEnvironmentName()`	Invalid state is rejected at the boundary with a clear message, never deep inside a test.
Caching (session reuse)	`auth.setup.ts` + `storageState`	An expensive result (logging in) is computed once and reused, with a fallback when it is stale.
Arrange–Act–Assert	All specs	Consistent test shape: setup via fixtures, one business action, then assertions.

> **CAREFUL** Don't claim patterns that aren't there. There is **no Screenplay pattern**, no **Builder class** (the data builder is a function), and no **Abstract Factory**. Naming the patterns accurately, and saying why you didn't need the others, comes across as more senior than listing many.

Why POM + fixtures rather than Screenplay?

Screenplay (Actors, Tasks, Questions) scales well for very large suites with many user roles, but adds a lot of boilerplate. With a handful of pages and one main user role, POM plus Playwright fixtures gives most of the benefit (reuse, readability, injection) with far less code. The fixture layer is also where Screenplay-style "abilities" would plug in if the suite grew.

7. Feature-by-feature walkthrough

For each feature: what it is, how it's built, where to find it, and how to describe it in an interview.

7.1 Page objects and components

How: one class per page extending `BasePage`; locators as `readonly` fields set in the constructor; actions as async methods; repeated UI (the left menu) as a component. Locators use stable ids and roles; ids containing dots (`customer.firstName`) use `[id="..."]` because `#customer.firstName` would be read as a class selector.

> **SAY** *"Locators live in exactly one place. When the Find Transactions page changed, the fix was in one page object, not in every test."*

7.2 Custom fixtures and the `loggedIn` state

How: `base.extend<Fixtures>()` registers every page object, the component and the API client. `loggedIn` opens Accounts Overview; if the Log Out link isn't visible (no session or expired), it logs in through the UI, then asserts the overview heading with a descriptive failure message.

7.3 Log in once: setup project + `storageState`

How: a `setup` project runs `tests/auth.setup.ts` once and saves cookies; every browser project declares `dependencies: ['setup']`; Account Services specs use `test.use({ storageState: AUTH_FILE })`. Session cookies are browser-independent, so **one login serves Chromium, Firefox and WebKit**.

Result: 24 logins → 1 per three-browser run. The saved file is git-ignored.

7.4 Configuration, environments and secrets

How: `env.ts` resolves values in priority order: real environment variables, then `.env.<TEST_ENV>`, then `.env`, then code defaults. `TEST_ENV` picks an entry from `ENVIRONMENTS`. Secrets have no defaults in code; locally they live in git-ignored `.env`, in CI in GitHub Secrets. `TEST_ENV` is used rather than `ENV` because some POSIX shells reserve `ENV`.

7.5 Production safety with tags

How: data-changing tests are tagged `@creates-account` or `@modifies-data`. `playwright.config.ts` sets `grepInvert` to exclude them when the environment has `allowDataChanges: false` (prod). Verified: 20 tests in qa, 16 in prod.

7.6 Self-contained test data

How: usernames are `john123_<base36 timestamp><random>`; transfer amounts come from `uniqueAmount()`, so a search finds *this* test's transaction. The duplicate-username test registers its own user first, then repeats, so it works on a fresh database and on the shared public one.

7.7 API testing

How: `LoginApi` wraps `GET /services/bank/login/{user}/{pass}` with URL encoding and `Accept: application/json`; tests assert status codes and the customer shape. API and UI share config and fixtures.

7.8 Rate-limit-aware execution

How: Chromium only by default (`ALL_BROWSERS=1` to add others), 2 workers on public environments, a single login per run, and a private Docker ParaBank (4 workers) for heavy runs. An Allure category groups 1015/429 failures so the cause is obvious.

7.9 Reporting: Playwright HTML + Allure

- **Business steps:** page actions are wrapped in `test.step()` (passwords never appear in step names).

- **Automatic Epic/Feature/Story:** an `auto: true` fixture derives labels from the spec path, for example `tests/ui/account-services/transfer-funds.spec.ts` → UI › Account Services › Transfer Funds.
- **Failure categories:** rate limited, login/credentials, timeouts, assertion failures, test code errors.
- **Environment panel** and an `environment` parameter on every test.
- **Trend history:** `scripts/allure-report.mjs` copies the previous `history/` into new results before generating.
- **Single-file report:** also built with `--single-file` so a downloaded CI artifact opens by double-click (the normal report shows "500 Failed to fetch" from `file://`).

7.10 Failure evidence

Evidence	Policy	Reason
Screenshot	only-on-failure	Cheap, instantly readable
Trace	retain-on-failure	Full timeline (DOM, network, console, screenshots) for almost no run-time cost
Video	on-first-retry	Recording every test added about 50% to run time; traces already cover the need

7.11 Docker

How: the `Dockerfile` starts from `mcr.microsoft.com/playwright:v1.65.0-noble` (browsers and OS libraries preinstalled) and caches the `npm ci` layer. `docker-compose.yml` runs a `parabank` service plus a `tests` service that waits for ParaBank (`wait-for-url.mjs`), runs with 4 workers, and copies reports to mounted folders **once at the end**.

Result: writing reports directly into the Windows/OneDrive bind mount made runs about 3× slower (≈60 s); writing inside the container and copying out gives ≈20 s.

7.12 CI/CD with GitHub Actions

How: triggers on push to `main`, pull requests and manual runs (with a qa/staging/prod picker). The job runs **inside the Playwright container** (no browser-install step), caches npm, reads credentials from secrets, sets up Java only after the tests (for Allure), and uploads the Allure single-file and HTML reports as artifacts. Baseline before containerising: 55 s total, of which 21 s was browser installation.

7.13 AI test agents

How: `npx playwright init-agents --loop=claude` generated planner, generator and healer agents plus an MCP server config. They were customised with project conventions (import fixtures from `baseTest`, keep locators in page objects, tag data-changing tests) and with logged-out and logged-in **seed** tests. `PW_AGENTS=1` (set in `.mcp.json`) makes seeds visible to agents but excludes them from normal runs.

7.14 Documentation and repository hygiene

README with setup, configuration, running, Docker, CI, coverage, contribution guide, design decisions and troubleshooting. `.gitignore` / `.dockerignore` exclude secrets, sessions and reports. Work on feature branches merged through pull requests.

8. Problems solved (STAR stories)

Interviewers remember stories. Each one below is real and backed by numbers from this project.

8.1 The suite collapsed when run in all browsers

Situation: running the full suite in three browsers produced 41 failures: timeouts, missing elements, HTTP 429. **Task:** find the cause rather than add retries. **Action:** grouped the failures by error message and read the failure snapshots: they showed Cloudflare's "Error 1015 — You are being rate limited". About 70 parallel sessions against one public server looked like a bot. Reduced the load at the source: login once with `storageState` (24 → 1 logins), Chromium by default, 2 workers, seeds excluded, plus a private Docker ParaBank for heavy runs. **Result:** 20/20 green in CI, and a dedicated Allure category so the problem is recognised instantly if it recurs.

8.2 A test that only passed by luck

Situation: "registering an existing username" passed on the public site but failed on a fresh Docker database. **Action:** traced it to a hidden dependency: it assumed `john123` already existed. Rewrote it to create its own unique user, log out, then register the same name again. **Result:** a deterministic test that works on any database. *Lesson: tests must own their data.*

8.3 Docker was slower than the laptop

Situation: the first Docker run took ≈60 s, slower than running locally. **Action:** measured instead of guessing. ParaBank answered in ~20 ms, so the server wasn't the cause. Running the same image *without* volume mounts took 19 s, which pointed to the Windows bind mount. Changed the container to write reports internally and copy them out once. Also found that video for every test cost ~50% and Windows `localhost` cost ~7 s. **Result:** ≈20 s in Docker; the Login/Register subset (8 runs incl. setup) went from 32.7 s to 16.3 s locally.

8.4 "500 Failed to fetch" on the downloaded report

Situation: the Allure artifact downloaded from CI showed an error. **Action:** reproduced it by opening both report types from `file://` in Chromium: browsers block a local page from fetching its own data files. **Result:** the pipeline now also produces a single-file report (verified: all 20 tests, no console errors) and uploads that one.

8.5 Secrets in a public repository

Situation: the repository is public and the tests need credentials. **Action:** removed all password defaults from code; loaded them from git-ignored `.env` locally and GitHub Secrets in CI; rejected workflow inputs for credentials because they're visible on the run page; added fail-fast checks with actionable messages. **Result:** no secret in the repository history, and a clear error instead of a confusing login failure when one is missing.

9. Lead-level presentation

9.1 The 2-minute pitch

> *"I built a Playwright and TypeScript framework for ParaBank, a demo online bank, covering both the UI and the REST API.*
>
> **Structure:** *it's layered: specs describe behaviour, page objects and a shared menu component own the locators, custom fixtures inject everything a test needs, and a typed config layer decides the environment and loads secrets from* `.env` *or GitHub Secrets.*
>
> **Reliability:** *the interesting part was making it reliable against a shared, rate-limited public server. When the full cross-browser run started failing, I traced it to Cloudflare rate limiting and redesigned the execution: one login reused through* `storageState` *across all browsers, per-environment worker limits, and a private ParaBank in Docker for heavy runs. Tests create their own unique data, so they run in parallel and on any database.*
>
> **Delivery:** *it runs locally, in Docker, and in GitHub Actions inside the Playwright container, with an environment picker where prod automatically skips data-changing tests. Reports are Playwright HTML plus Allure with business-level steps, automatic Epic/Feature/Story labels, failure categories and trends.*
>
> **AI:** *I also set up Playwright's planner, generator and healer agents, tailored to the framework's conventions."*

9.2 The 10-minute walkthrough (with the repository open)

Min	Show	Talk about
0–1	README, folder tree	Goal, scope, layers
1–3	A spec → `baseTest.ts` → a page object	How a test reads; DI via fixtures; encapsulated locators; `test.step()`
3–4	`env.ts`, `environments.ts`	Typed environments, secrets priority order, type guard, fail fast
4–5	`auth.setup.ts`, config projects	Login once, fallback login, the rate-limit story
5–6	Tags + `grepInvert`	Prod safety; self-contained data
6–7	`npx playwright test --ui`, a trace	Live run; debugging with traces
7–8	Allure report	Behaviors tab, steps, categories, trends
8–9	Dockerfile, compose, workflow	Reproducibility; measured speed-ups; CI secrets and artifacts
9–10	Agents, roadmap	AI-assisted planning and healing; what's next

9.3 What makes it "lead level"

- **Diagnosis over workarounds:** rate limits and slow Docker runs were measured and fixed at the root, not hidden with retries.
- **Designing for the team:** conventions are documented in the README and encoded in the AI agents, so new contributors (human or AI) follow the same structure.
- **Safety rails:** prod protection, secret handling and fail-fast config prevent costly mistakes.
- **Developer experience:** one-command runs (`npm run docker:test`), readable reports, and double-click artifacts.
- **Process:** feature branches, pull requests and CI on every change.

10. Architect-level discussion

10.1 Key decisions and trade-offs

Decision	Benefit	Trade-off / mitigation
Playwright for UI + API	One runner, one language, shared fixtures and config	Very heavy API suites might prefer a dedicated tool; current scope doesn't need one
POM + fixtures (not Screenplay)	Low boilerplate, easy onboarding	Revisit if many user roles appear; fixtures are the extension point
Shared login session	24 → 1 logins, faster, kinder to the server	Tests share one user; mitigated by unique data and a fallback login
Chromium-only default	Fast, avoids rate limits	Cross-browser on demand (`ALL_BROWSERS=1`), suited to nightly runs
Video only on retry	~50% faster runs	No video on a first failure; trace + screenshot cover it
Private ParaBank in Docker	No rate limit, isolated data, safe for destructive tests	Not production-like; public/qa runs remain the reference
CI job in the Playwright container	No browser install; identical to the local Docker runner	Image pull time; tag must match the npm version
Secrets via .env / GitHub Secrets	Nothing sensitive in a public repo	Per-environment secrets next: GitHub Environments with approval on prod

10.2 Scaling the framework

- **More modules:** one page object + one fixture line + a spec folder; Allure labels and conventions apply automatically. The planner/generator agents speed this up.
- **More tests:** sharding (`--shard=1/4`) across CI runners; a matrix job per browser; workers tuned per environment.
- **More teams:** shared fixtures/pages as an internal package; CODEOWNERS on `src/`; lint and type-check as merge gates.
- **Faster, more stable data setup:** create accounts and transactions through the API before UI tests, instead of through the UI.
- **More environments:** one entry in `ENVIRONMENTS` + an optional `.env.<env>`; GitHub Environments for per-environment secrets and approvals.

10.3 Quality gates and governance

- CI on every pull request; `forbidOnly` in CI blocks accidental `test.only`.
- Retries only in CI (2), with video on the first retry, so flaky tests are visible rather than hidden.
- Strict TypeScript; next step: `tsc --noEmit`, ESLint and Prettier as required CI checks.
- Reports as the shared source of truth: Allure for stakeholders, Playwright traces for developers.

10.4 Metrics to track

Metric	Why
Pass rate and flaky rate (Allure trends, retries)	Trust in the suite
Suite duration per environment	Feedback speed; regression in framework performance
Failures by category	Environment problems vs real product bugs vs test-code bugs
Coverage by module (Allure Behaviors)	Where tests are missing

10.5 Risks and mitigations

Risk	Mitigation
Shared public data changes or resets	Self-contained unique data; private Docker instance
Rate limiting	Single login, low workers, Docker for load, dedicated failure category
Locator drift after UI changes	Locators centralised in page objects; healer agent; traces for diagnosis
Accidental changes in prod	Tags + automatic `grepInvert`; planned approval gate on prod
Version drift (image vs npm package)	Pinned image tag with a comment to keep it in sync with `@playwright/test`

10.6 Roadmap

1) ESLint + Prettier + type-check in CI · 2) broader API layer (accounts, transactions, transfer) with typed models and JSON-schema checks · 3) API-driven test data for UI tests · 4) GitHub Environments with prod approval · 5) per-browser CI matrix and sharding · 6) visual (`toHaveScreenshot`) and accessibility (axe) checks · 7) more modules (Bill Pay, Open New Account, Request Loan) via the agents.

11. Interview questions & answers

Why Playwright over Selenium?

Auto-waiting locators remove most explicit waits; one tool covers UI and API; parallelism, fixtures, traces and multi-browser are built in; and it's fast because it drives browsers over a direct protocol rather than WebDriver.

How do you handle authentication?

A setup project logs in once and saves `storageState`; browser projects depend on it; Account Services specs load the saved state. The `loggedIn` fixture checks for the Log Out link and falls back to a UI login if the session is missing or expired.

How do you manage environments and secrets?

`TEST_ENV` selects a typed environment config. Values resolve from real environment variables, then `.env.<env>`, then `.env`, then code defaults. Secrets exist only in git-ignored `.env` and GitHub Secrets; a missing one fails fast with an actionable message.

How do you stop tests damaging production?

Data-changing tests carry `@creates-account` or `@modifies-data`. For any environment with `allowDataChanges: false`, the config sets `grepInvert` so those tests aren't even collected.

How do you handle flaky tests?

First find the cause: traces, categories and trends. Then fix it at the source: unique data, web-first assertions, no fixed sleeps, controlled load. Retries exist only in CI, and with video on the first retry, so flakiness stays visible.

How do you make tests independent?

Each test creates the data it relies on, with unique values (base-36 timestamp + random suffix, unique amounts), so tests can run in any order, in parallel, on a fresh or shared database.

What are fixtures and why use them over beforeEach?

Fixtures are typed, lazily created, composable dependencies with built-in setup and teardown. Tests declare only what they need, and fixtures can be overridden per file (`test.use`). `beforeEach` runs everything for every test and isn't reusable across files.

How would you speed up the suite?

Measure first: per-test durations and step timings. In this project that found video recording (~50%), the Windows `localhost` delay, slow bind mounts in Docker and repeated logins. Then reduce work (one login, video on retry), parallelise where the server allows (workers per environment, sharding), and remove setup cost in CI (pre-built Playwright image).

How do you debug a CI-only failure?

Download the artifacts: the single-file Allure report for the overview, and the Playwright trace for the step-by-step DOM, network and console. Check the failure category (rate limit vs assertion vs credentials). Reproduce with the same image locally via Docker.

What would you change if the suite had 2,000 tests?

Shard across runners; split smoke/regression with tags; create data through the API; shared components as an internal package; strict lint/type gates; ownership per module; nightly cross-browser runs and fast PR runs.

Where are the OOP concepts in your framework?

Encapsulation in page objects and the `ENV` getters; abstraction through the abstract `BasePage` and business-level methods; single-level inheritance from `BasePage`; composition for the menu component and fixtures; polymorphism through subtype substitution and interchangeable environment configs.

Which design patterns did you use, and why not Screenplay?

POM, Component Object, DI via fixtures, Factory, Test Data Builder, Facade, module Singleton for config, Strategy for environments, and wrappers via `test.step`. Screenplay would add a lot of structure for a suite with few pages and one main role; fixtures leave room to adopt it later.

How did you use AI in this project?

As an accelerator under my direction: generating boilerplate, the Playwright planner/generator/healer agents for test planning and locator repair, and reviewing changes. I set the architecture, the conventions the agents must follow, and verified every result by running it. Every number in this framework was measured.

12. Command cheat sheet & file map

Command	Purpose
`npm test`	All tests, default `qa`, Chromium
`npm run test:local | test:qa | test:staging | test:prod`	Run against an environment
`npm run test:all-browsers`	Chromium + Firefox + WebKit, one worker
`npx playwright test --ui`	Interactive UI mode
`npx playwright test --debug`	Step-by-step Inspector
`npx playwright test -g "name"`	Run tests matching a title
`npx playwright codegen <url>`	Record actions / pick locators
`npm run docker:test`	ParaBank + tests in Docker, reports copied back
`npm run docker:parabank` / `docker:down`	Start / stop the private ParaBank
`npm run report`	Playwright HTML report
`npm run allure:report`	Build (with history) and open Allure
`npm run agents:init`	Regenerate the Playwright AI agents

File map

File	Responsibility
`playwright.config.ts`	Projects, workers, prod filter, reporters, evidence policy
`src/config/env.ts`	Loads .env files, validates TEST_ENV, exposes `ENV`
`src/config/environments.ts`	Typed environment definitions
`src/config/auth.ts`	Saved-session file path
`src/fixtures/baseTest.ts`	Fixtures, `loggedIn`, automatic Allure labels
`src/pages/*.ts`	Page objects (BasePage + 5 pages)
`src/components/AccountServicesMenu.ts`	Shared menu component, typed link names
`src/api/LoginApi.ts`	REST client for login
`src/utils/*.ts`	Dates, unique amounts, random strings
`tests/auth.setup.ts`	One-time login, saves session
`tests/ui/**, tests/api/**`	Specs by layer and module
`test-data/*.json`	Default data and expected messages
`scripts/allure-report.mjs`	Allure with history + single-file report
`scripts/wait-for-url.mjs`	Waits for ParaBank in Docker
`Dockerfile`, `docker-compose.yml`	Runner image; ParaBank + tests
`.github/workflows/playwright.yml`	CI pipeline
`.claude/agents/*`, `.mcp.json`	Playwright AI agents and their browser server